Лабораторна робота №1. Auth-сервіс і Catalog-сервіс

Мета: винести з монолітного застосунку два перші сервіси — автентифікації (користувачі, ролі, JWT) і каталогу (позиції, які можна замовити); налаштувати в кожному з них локальну перевірку токена; перемкнути відповідні маршрути шлюзу з моноліту на нові сервіси, не змінюючи клієнта.

1. Що саме робимо

У ЛР0 ви поставили перед монолітом шлюз і описали, як саме розділяєте систему. Тепер виконуєте перші два кроки цього плану. Моноліт продовжує обслуговувати решту функцій — так і має бути.

Було після ЛР0	Стане після ЛР1
Клієнт → Шлюз → Моноліт (усе)	Клієнт → Шлюз → **AuthService** (`/api/v1/auth/**`, `/api/v1/users/**`)
	Клієнт → Шлюз → **CatalogService** (`/api/v1/catalog/**`)
	Клієнт → Шлюз → Моноліт (усе інше)

Узагальнені назви, як завжди, підставляєте свої: **Item** — це вистава, сеанс, подія, товар або послуга вашої предметної області.

1.1. Домовленість щодо баз даних

Діє принцип **database per service**: кожен сервіс володіє власною базою даних, і ніхто інший до неї не звертається.

- дві нові бази: `miniproject_auth` і `miniproject_catalog`. Тримати їх на одному сервері СУБД разом із базою моноліту — нормально; головне, що це **різні бази з різними рядками підключення**;
- у кожного сервісу — **власний DbContext** і **власний набір міграцій**;
- **заборонено** зовнішні ключі та `JOIN` між базами різних сервісів; зв’язок лише за ідентифікатором;
- **заборонено** підключатися з одного сервісу до бази іншого (зокрема до бази моноліту). Потрібні чужі дані — звертайтеся до API власника;
- дані каталогу переносяться з моноліту **копіюванням** (скрипт або ручний експорт), а не спільним доступом до тієї самої таблиці.

> **Важливо:** Після цієї роботи в системі якийсь час існують дві «правди» про каталог: стара таблиця в моноліті й нова база Catalog-сервісу. Це нормальний стан переходу. Правило просте: щойно маршрут перемкнено на сервіс — **єдиним власником даних стає сервіс**, а таблиця моноліту вважається застарілою і більше не змінюється.

2. Методичні відомості

2.1. Роль Auth-сервісу в системі

Автентифікація централізована, а перевірка прав — розподілена. Токени видає тільки Auth-сервіс. Усі інші сервіси лише перевіряють підпис отриманого токена, і роблять це **локально, без мережевого запиту**:

```
Клієнт ──POST /api/v1/auth/login──▶ Gateway ──▶ AuthService ──▶ JWT

Клієнт ──GET /api/v1/catalog/items──▶ Gateway ──▶ CatalogService
       Authorization: Bearer <JWT>                    │
                                                      └─ перевірка підпису
                                                         ключем із конфігурації
```

Тому `Jwt:Key`, `Jwt:Issuer` та `Jwt:Audience` мають бути **однаковими** в обох сервісах. Розійдуться — Catalog не прийме токен, виданий Auth.

2.2. Перевірка токена (однаково в обох сервісах)

```
var jwt = builder.Configuration.GetSection("Jwt");

builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            ValidIssuer = jwt["Issuer"],
            ValidAudience = jwt["Audience"],
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes(jwt["Key"]!)),
            ClockSkew = TimeSpan.FromSeconds(30)
        };
    });

builder.Services.AddAuthorization();
// ...
app.UseAuthentication();   // спочатку «хто ти»,
app.UseAuthorization();    // потім «що тобі можна»
```

> **Порада:** Цей блок потрібен **обом** сервісам, а далі знадобиться й Booking-сервісу. Не копіюйте його втретє: винесіть у спільний проєкт-бібліотеку `Shared` як метод-розширення `AddJwtAuth(this IServiceCollection, IConfiguration)`. Пам’ятайте правило з лекції: у `Shared` можна виносити наскрізні дрібниці, але не сутності й не бізнес-логіку.

2.3. Видача токена

Каркас; список claims і час життя визначаєте самостійно.

```
var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_options.Key));
var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

var claims = new List<Claim>
{
    new(JwtRegisteredClaimNames.Sub, user.Id.ToString()),
    new(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString()),
    // ... решту claims додайте самостійно (email, ролі, ім'я)
};

var token = new JwtSecurityToken(
    issuer: _options.Issuer, audience: _options.Audience, claims: claims,
    expires: DateTime.UtcNow.AddMinutes(_options.AccessTokenMinutes),
    signingCredentials: creds);

return new JwtSecurityTokenHandler().WriteToken(token);
```

Ключ для HMAC-SHA256 має бути **не коротшим за 32 байти**. Ролі додаються claim-ом `ClaimTypes.Role` — по одному на кожну роль.

2.4. Refresh-токени

Access-токен навмисно короткоживучий (10–20 хвилин): відкликати виданий JWT неможливо, тому чим коротше він живе, тим менша шкода від викрадення. Щоб користувач не вводив пароль щочверть години, разом із ним видається **refresh-токен** — довгоживучий випадковий рядок, який зберігається в базі й може бути відкликаний.

1. вхід → пара `access` + `refresh`;
2. access спливає → клієнт викликає `POST /auth/refresh`;
3. сервіс перевіряє, що токен існує, не прострочений і не відкликаний;
4. видає **нову пару**, старий refresh позначає використаним (**ротація**);
5. при виході refresh позначається відкликаним.

Refresh-токен генеруйте через `RandomNumberGenerator`, а не `Guid.NewGuid()` — і будьте готові пояснити чому.

2.5. Catalog-сервіс: що саме виноситься

З моноліту переїжджає контролер каталогу разом зі своїми сутностями. Це найпростіший для винесення сервіс: він майже нічого не потребує від інших — тільки перевіряє токен, коли дані змінюють.

Три речі, які треба зробити краще, ніж було в моноліті:

1. **DTO замість сутностей.** Назовні не виходить жоден клас EF Core. Клієнт не має бачити ані службових полів, ані навігаційних властивостей.
2. **Пагінація, фільтрація, сортування** у списковому ендпоінті. Віддавати клієнтові «усі 10 000 позицій» — помилка проєктування.
3. **Розділення прав:** читання доступне всім (або всім автентифікованим — вирішуєте ви), а створення, зміна й видалення — лише ролі `Admin`.

> **Пагінація —** віддавання даних сторінками: клієнт указує номер сторінки й розмір, сервіс повертає лише потрібний фрагмент і загальну кількість записів.

```
// приклад відповіді спискового ендпоінта
{
  "items": [ { "id": 1, "title": "..." } ],
  "page": 1,
  "pageSize": 20,
  "totalCount": 137
}
```

Обмежте `pageSize` згори (наприклад, 100) — інакше клієнт однаково зможе попросити все одразу.

2.6. Валідація вхідних даних

Атрибут `[ApiController]` вмикає автоматичну перевірку моделі: некоректний DTO дає `400 Bad Request` у форматі `ValidationProblemDetails` без жодного рядка вашого коду.

```
public class CreateItemRequest
{
    [Required, MaxLength(200)]
    public string Title { get; set; } = null!;

    [Range(0, 1_000_000)]
    public decimal Price { get; set; }
    // ...
}
```

Складніші правила («дата початку не в минулому», «категорія існує») зручніше описати бібліотекою `FluentValidation` — це вітається, але не обов'язково.

2.7. Документування OpenAPI

З .NET 9 шаблон Web API не містить Swashbuckle: специфікацію генерує вбудований `Microsoft.AspNetCore.OpenApi` (`AddOpenApi()` / `MapOpenApi()`), а інтерфейс підключається окремо — `Scalar.AspNetCore` або `Swashbuckle.AspNetCore.SwaggerUI`. Щоб у ньому працювала кнопка **Authorize**, у документ треба додати схему безпеки:

```
internal sealed class BearerSecuritySchemeTransformer : IOpenApiDocumentTransformer
{
    public Task TransformAsync(OpenApiDocument document,
        OpenApiDocumentTransformerContext context, CancellationToken ct)
    {
        document.Components ??= new OpenApiComponents();
        document.Components.SecuritySchemes["Bearer"] = new OpenApiSecurityScheme
        {
            Type = SecuritySchemeType.Http, Scheme = "bearer",
            In = ParameterLocation.Header, BearerFormat = "JWT"
        };
        // Доповніть: вимогу безпеки (SecurityRequirement) для захищених операцій
        return Task.CompletedTask;
    }
}
```

Повне документування означає: XML-коментарі увімкнено й підхоплено, кожен ендпоінт має `/// <summary>`, вказано всі коди відповіді через `[ProducesResponseType]`, описано поля DTO, працює авторизація в UI.

3. Підготовка проєктів

1. Створіть два проєкти й додайте їх до рішення:

```
dotnet new webapi -n AuthService    -o src/AuthService    --use-controllers
dotnet new webapi -n CatalogService -o src/CatalogService --use-controllers
dotnet sln add src/AuthService src/CatalogService
```

2. Зафіксуйте порти: Auth — 5001, Catalog — 5002 (шлюз уже працює на 5000, моноліт — на 5100).
3. У кожному проєкті створіть папки `Models`, `Data`, `Dtos`, `Repositories`, `Services`, `Controllers`, `Configuration`.
4. Підключіть пакети:

Пакет	Кому	Призначення
`Microsoft.AspNetCore.Authentication.JwtBearer`	обом	Перевірка JWT
`System.IdentityModel.Tokens.Jwt`	Auth	Генерація JWT
`Microsoft.EntityFrameworkCore` + `.Design`	обом	ORM і міграції
`Pomelo.EntityFrameworkCore.MySql`	обом	Провайдер MySQL (або аналог)
`Microsoft.AspNetCore.Identity` або `BCrypt.Net-Next`	Auth	Хешування паролів
`Microsoft.AspNetCore.OpenApi` + `Scalar.AspNetCore`	обом	OpenAPI та UI
`FluentValidation.AspNetCore` *(за бажанням)*	обом	Складна валідація

5. Створіть дві бази даних і пропишіть рядки підключення в конфігурації відповідних сервісів.

4. Auth-сервіс

4.1. Модель даних

Сутність	Поля (мінімум)	Примітки
`User`	`Id`, `Email`, `PasswordHash`, `DisplayName`, `CreatedAt`, `IsActive`	Унікальний індекс на `Email`
`Role`	`Id`, `Name`	Мінімум `User` і `Admin`; заповнюються під час seed
`UserRole`	`UserId`, `RoleId`	Зв’язок «багато-до-багатьох»
`RefreshToken`	`Id`, `UserId`, `Token`, `ExpiresAt`, `CreatedAt`, `RevokedAt`, `ReplacedByToken`	Зберігається в базі, бо саме його можна відкликати

Завдання щодо класу AuthDbContext: налаштувати підключення до `miniproject_auth`; додати `DbSet` для всіх сутностей; в `OnModelCreating` — унікальний індекс на `Email`, зв'язок «багато-до-багатьох», значення за замовчуванням для `CreatedAt`, максимальні довжини рядків; передбачити seed ролей та одного адміністратора; створити й застосувати міграцію.

4.2. Контракт API

Метод і маршрут	Доступ	Призначення	Коди
`POST /api/v1/auth/register`	анонім	Реєстрація (роль `User` за замовчуванням)	201, 400, 409
`POST /api/v1/auth/login`	анонім	Вхід; повертає пару токенів	200, 400, 401
`POST /api/v1/auth/refresh`	анонім	Оновлення пари токенів	200, 400, 401
`POST /api/v1/auth/logout`	авториз.	Відкликання refresh-токена	204, 401
`GET /api/v1/users/me`	авториз.	Профіль поточного користувача	200, 401
`GET /api/v1/users`	`Admin`	Список користувачів із пагінацією	200, 401, 403
`PUT /api/v1/users/{id}/roles`	`Admin`	Зміна набору ролей	204, 400, 401, 403, 404
`GET /health`	анонім	Стан сервісу та з'єднання з БД	200, 503

> **Увага:** `GET /users/me` не приймає ідентифікатор у запиті — він береться **з токена**: `User.FindFirstValue(ClaimTypes.NameIdentifier)`.

4.3. Завдання на реалізацію

Нижче окреслено *призначення* класів. Склад методів, властивостей і залежностей визначаєте самостійно — це оцінювана частина роботи.

- **Репозиторії:** узагальнений `IBaseRepository<T>` і специфічні `IUserRepository` (пошук за email, перевірка існування, завантаження користувача разом із ролями через `Include`), `IRefreshTokenRepository`.
- **TokenService:** відповідає **лише** за токени — складання claims, генерація access і refresh, обчислення часів. Подумайте, чому цей клас не має жодного разу звертатися до `DbContext`.
- **AuthService / UserService:** сценарії реєстрації, входу, оновлення й відкликання токенів, керування ролями. Вирішіть, як сервіс повідомляє контролер про невдачу — винятком, `null` чи об'єктом-результатом; той самий підхід застосуєте в Catalog.
- **Контролери** — прийняти DTO, викликати сервіс, повернути коректний статус-код. Жодної бізнес-логіки й жодного `DbContext`.

5. Catalog-сервіс

5.1. Модель даних

Сутність	Поля (мінімум)	Примітки
`Item`	`Id`, `Title`, `Description`, `Price`, `CategoryId`, `IsActive`, `CreatedAt`	Ваша основна сутність каталогу: подія, сеанс, товар
`Category`	`Id`, `Name`	Довідник; за потреби замініть на власну структуру
(ваші додаткові)	…	Місткість, дата проведення, зображення — залежно від предметної області

Завдання щодо класу CatalogDbContext: підключення до `miniproject_catalog`; конфігурація полів (довжини, точність `decimal` для ціни); індекс за полем, за яким найчастіше шукають; seed кількох позицій для перевірки; міграція.

Дані з моноліту перенесіть одноразово — SQL-скриптом або невеликою консольною утилітою.

5.2. Контракт API

Метод і маршрут	Доступ	Призначення	Коди
`GET /api/v1/catalog/items`	анонім	Список із пагінацією, пошуком і сортуванням	200, 400
`GET /api/v1/catalog/items/{id}`	анонім	Одна позиція	200, 404
`POST /api/v1/catalog/items`	`Admin`	Створення позиції	201, 400, 401, 403
`PUT /api/v1/catalog/items/{id}`	`Admin`	Повне оновлення	204, 400, 401, 403, 404
`DELETE /api/v1/catalog/items/{id}`	`Admin`	Видалення (краще «м’яке» — `IsActive = false`)	204, 401, 403, 404
`GET /api/v1/catalog/categories`	анонім	Довідник категорій	200
`GET /health`	анонім	Стан сервісу та з’єднання з БД	200, 503

Параметри спискового ендпоінта (мінімум): `page`, `pageSize`, `search`, `categoryId`, `sortBy`, `desc`. Некоректні значення (`page = 0`, `pageSize = 10000`) мають давати `400`, а не падіння.

5.3. Завдання на реалізацію

- **Репозиторій каталогу:** метод, що повертає сторінку результатів разом із загальною кількістю. Фільтрація й сортування мають виконуватися **в базі даних**, а не після завантаження всіх записів у пам’ять — будьте готові показати згенерований SQL.
- **Сервіс каталогу:** перевірки предметної області (наприклад, чи існує категорія), перетворення сутностей на DTO, «м’яке» видалення.
- **DTO:** окремі для читання (`ItemDto`, `ItemDetailsDto`) і для запису (`CreateItemRequest`, `UpdateItemRequest`). Подумайте, які поля не повинні прийматися від клієнта.

- **Захист:** атрибути `[Authorize(Roles = "Admin")]` на змінювальних операціях. Перевірте, що сервіс сам приймає токен, виданий Auth-сервісом, — без жодного звернення до нього.
- **Ваше завдання на подумати:** що робити, якщо адміністратор змінив ціну позиції, а вона вже потрапила до чийогось замовлення в моноліті. Відповідь запишіть у звіт — на ЛР2 ця тема стане практичною.

6. Перемикання шлюзу

Найприємніша частина роботи: клієнт нічого не помічає, а трафік уже йде на нові сервіси. У `ocelot.json` змінюються лише адреси у двох маршрутах, які ви підготували в ЛР0.

```
{
  "Routes": [
    {
      "UpstreamPathTemplate": "/api/v1/auth/{everything}",
      "DownstreamPathTemplate": "/api/v1/auth/{everything}",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [ { "Host": "localhost", "Port": 5001 } ]
    },
    {
      "UpstreamPathTemplate": "/api/v1/catalog/{everything}",
      "DownstreamPathTemplate": "/api/v1/catalog/{everything}",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [ { "Host": "localhost", "Port": 5002 } ]
    },
    {
      "UpstreamPathTemplate": "/api/{everything}",
      "DownstreamPathTemplate": "/api/{everything}",
      "DownstreamScheme": "http",
      "DownstreamHostAndPorts": [ { "Host": "localhost", "Port": 5100 } ]
    }
  ]
}
```

Порядок маршрутів має значення: конкретні — вище, перехоплювач на моноліт — останнім. Не забудьте вказати `UpstreamHttpMethod` там, де потрібно обмежити методи.

- **Завдання:** переконайтеся, що старі маршрути моноліту працюють, як і раніше, а два перемкнені — уже на нових сервісах. Різницю має бути видно в логах шлюзу.

7. Обов’язкові вимоги до сервісів

Вимоги наскрізні: вони перевіряються в цій і в усіх наступних роботах, **для кожного** сервісу.

1. **Валідація вхідних даних:** некоректний запит повертає `400` з описом полів, а не `500`.
2. **Ізоляція моделі:** сутності EF Core не з’являються в сигнатурах контролерів — тільки DTO.
3. **Повне документування OpenAPI:** summary, коди відповідей, описи DTO, робоча авторизація в UI.
4. **Уніфікована обробка помилок:** глобальний обробник, `ProblemDetails`, жодних stack trace у відповіді.

5. **Логування:** `ILogger` у сервісах; фіксуються вхід, невдалі спроби входу, зміни каталогу, помилки. Паролі й токени в логи не потрапляють.
6. **Конфігурація без секретів у коді:** усі параметри — з конфігурації, user-secrets або змінних середовища.
7. **Версіонування API:** усі маршрути під `/api/v1/`.
8. **Розширюваність:** робота через інтерфейси й впровадження залежностей.
9. **Health check:** `/health`, що перевіряє і застосунок, і доступність БД.
10. **Автономність:** кожен сервіс запускається окремо, на власному порту, і не залежить від коду інших сервісів під час компіляції.

8. Порядок перевірки роботи

Усі запити виконуйте **через шлюз** (`http://localhost:5000`) — саме так це побачить клієнт:

1. усі три застосунки запущено; `/health` кожного відповідає `200`;
2. реєстрація нового користувача → `201`; повторна з тим самим email → `409`; короткий пароль → `400` з переліком полів;
3. у таблиці `Users` бази `miniproject_auth` пароль збережено як хеш (покажіть запис у СУБД);
4. вхід із правильним паролем → `200` і пара токенів; з неправильним → `401`;
5. вміст токена розібрано на jwt.io — claims відповідають користувачеві;
6. `GET /api/v1/users/me` без токена → `401`, з токеном → `200`;
7. `GET /api/v1/catalog/items?page=1&pageSize=5&search=...` повертає сторінку з `totalCount`;
8. `POST /api/v1/catalog/items` без токена → `401`; з токеном звичайного користувача → `403`; з токеном адміністратора → `201`;
9. запит до каталогу видно в логах **Catalog-сервісу**, а не моноліту; запит до будь-якого іншого маршруту — у логах моноліту;
10. після завершення терміну дії access-токена запит → `401`, а `POST /auth/refresh` видає нову пару; після `logout` той самий refresh більше не приймається.

9. Контрольні питання

1. Чим автентифікація відрізняється від авторизації? Який із процесів у нашій системі централізований?
2. З яких частин складається JWT? Що саме захищає підпис і чого він не захищає?
3. Чому Catalog-сервіс не звертається до Auth-сервісу, щоб перевірити токен?
4. Що станеться, якщо `Jwt:Key` у двох сервісах відрізнятиметься? А `Jwt:Audience`?
5. Чому пароль не можна зберігати як хеш SHA-256 без солі?
6. Навіщо потрібен refresh-токен і що таке його ротація?
7. Чим відрізняються коди 401 і 403? Наведіть приклад для кожного зі своїх сервісів.

8. Чому не можна віддавати клієнту сутність EF Core замість DTO?
9. Як реалізовано пагінацію у вашому каталозі? Що повертається клієнтові, крім самих даних?
10. Чому фільтрація має відбуватися в базі даних, а не в пам'яті? Як ви це перевірили?
11. Чому кожен сервіс має власну базу даних? Що саме це забороняє?
12. Які рядки в `ocelot.json` ви змінили і чому порядок маршрутів має значення?
13. Як тепер виглядає шлях запиту `GET /api/v1/catalog/items` від клієнта до бази даних?
14. Що станеться зі старою таблицею каталогу в моноліті і чому її більше не можна змінювати?